

07-Control-Flow-Statements

January 20, 2025

This notebook contains an excerpt from the [Whirlwind Tour of Python](#) by Jake VanderPlas; the content is available [on GitHub](#).

The text and code are released under the [CC0](#) license; see also the companion project, the [Python Data Science Handbook](#).

< [Built-In Data Structures](#) | [Contents](#) | [Defining and Using Functions](#) >

1 Control Flow

Control flow is where the rubber really meets the road in programming. Without it, a program is simply a list of statements that are sequentially executed. With control flow, you can execute certain code blocks conditionally and/or repeatedly: these basic building blocks can be combined to create surprisingly sophisticated programs!

Here we'll cover *conditional statements* (including “`if`”, “`elif`”, and “`else`”), *loop statements* (including “`for`” and “`while`” and the accompanying “`break`”, “`continue`”, and “`pass`”).

1.1 Conditional Statements: `if-elif-else`:

Conditional statements, often referred to as *if-then* statements, allow the programmer to execute certain pieces of code depending on some Boolean condition. A basic example of a Python conditional statement is this:

```
[1]: x = -15

if x == 0:
    print(x, "is zero")
elif x > 0:
    print(x, "is positive")
elif x < 0:
    print(x, "is negative")
else:
    print(x, "is unlike anything I've ever seen...")
```

-15 is negative

Note especially the use of colons (`:`) and whitespace to denote separate blocks of code.

Python adopts the `if` and `else` often used in other languages; its more unique keyword is `elif`, a contraction of “else if”. In these conditional clauses, `elif` and `else` blocks are optional; additionally,

you can optionally include as few or as many `elif` statements as you would like.

1.2 for loops

Loops in Python are a way to repeatedly execute some code statement. So, for example, if we'd like to print each of the items in a list, we can use a `for` loop:

```
[2]: for N in [2, 3, 5, 7]:  
      print(N, end=' ') # print all on same line
```

2 3 5 7

Notice the simplicity of the `for` loop: we specify the variable we want to use, the sequence we want to loop over, and use the “`in`” operator to link them together in an intuitive and readable way. More precisely, the object to the right of the “`in`” can be any Python *iterator*. An iterator can be thought of as a generalized sequence, and we'll discuss them in [Iterators](#).

For example, one of the most commonly-used iterators in Python is the `range` object, which generates a sequence of numbers:

```
[3]: for i in range(10):  
      print(i, end=' ')
```

0 1 2 3 4 5 6 7 8 9

Note that the range starts at zero by default, and that by convention the top of the range is not included in the output. Range objects can also have more complicated values:

```
[4]: # range from 5 to 10  
      list(range(5, 10))
```

```
[4]: [5, 6, 7, 8, 9]
```

```
[5]: # range from 0 to 10 by 2  
      list(range(0, 10, 2))
```

```
[5]: [0, 2, 4, 6, 8]
```

You might notice that the meaning of `range` arguments is very similar to the slicing syntax that we covered in [Lists](#).

Note that the behavior of `range()` is one of the differences between Python 2 and Python 3: in Python 2, `range()` produces a list, while in Python 3, `range()` produces an iterable object.

1.3 while loops

The other type of loop in Python is a `while` loop, which iterates until some condition is met:

```
[6]: i = 0  
      while i < 10:  
          print(i, end=' ')  
          i += 1
```

```
0 1 2 3 4 5 6 7 8 9
```

The argument of the `while` loop is evaluated as a boolean statement, and the loop is executed until the statement evaluates to `False`.

1.4 `break` and `continue`: Fine-Tuning Your Loops

There are two useful statements that can be used within loops to fine-tune how they are executed:

- The `break` statement breaks-out of the loop entirely
- The `continue` statement skips the remainder of the current loop, and goes to the next iteration

These can be used in both `for` and `while` loops.

Here is an example of using `continue` to print a string of odd numbers. In this case, the result could be accomplished just as well with an `if-else` statement, but sometimes the `continue` statement can be a more convenient way to express the idea you have in mind:

```
[7]: for n in range(20):  
      # if the remainder of n / 2 is 0, skip the rest of the loop  
      if n % 2 == 0:  
          continue  
      print(n, end=' ')
```

```
1 3 5 7 9 11 13 15 17 19
```

Here is an example of a `break` statement used for a less trivial task. This loop will fill a list with all Fibonacci numbers up to a certain value:

```
[8]: a, b = 0, 1  
     amax = 100  
     L = []  
  
     while True:  
         (a, b) = (b, a + b)  
         if a > amax:  
             break  
         L.append(a)  
  
     print(L)
```

```
[1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

Notice that we use a `while True` loop, which will loop forever unless we have a `break` statement!

1.5 Loops with an `else` Block

One rarely used pattern available in Python is the `else` statement as part of a `for` or `while` loop. We discussed the `else` block earlier: it executes if all the `if` and `elif` statements evaluate to `False`. The loop-`else` is perhaps one of the more confusingly-named statements in Python; I prefer to think of it as a `nobreak` statement: that is, the `else` block is executed only if the loop ends naturally, without encountering a `break` statement.

As an example of where this might be useful, consider the following (non-optimized) implementation of the *Sieve of Eratosthenes*, a well-known algorithm for finding prime numbers:

```
[9]: L = []
    nmax = 30

    for n in range(2, nmax):
        for factor in L:
            if n % factor == 0:
                break
        else: # no break
            L.append(n)
    print(L)
```

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

The `else` statement only executes if none of the factors divide the given number. The `else` statement works similarly with the `while` loop.

< [Built-In Data Structures](#) | [Contents](#) | [Defining and Using Functions](#) >